

THE PROTOCOL 01 MANIFESTO

PRIVACY LAYER FOR SOLANA

Every transaction you make on a public blockchain is a confession. Who paid you, who you paid, how much you hold, what you subscribe to, when you wake up and when you sleep. Solana settles in 400 milliseconds and remembers forever.

We think that is backwards. Cash never asked for your identity. Your bank statement was never a public document. Privacy is not a feature you bolt on for criminals, it is the default state of money that every ledger before the blockchain simply had.

Protocol 01 exists to give Solana that default back.

What we believe

Five convictions. Everything in the stack follows from them.

1 Privacy is normal

Wanting your salary, your savings, and your spending habits off a public feed is not suspicious. It is how money has always worked.

2 Privacy must be usable

A privacy tool that requires a desktop, a command line, and a cryptography degree protects nobody. If it does not run on a phone with one tap, it does not exist.

3 Privacy must be durable

A quantum computer built in 2035 can decrypt everything recorded in 2026. Privacy that expires is not privacy, it is a time bomb. That is why we build on hash-based cryptography that quantum computers do not break.

4 Trust nothing you can verify

No trusted setup ceremonies, no coordinator secrets, no "trust our server". Proofs are generated on your device and verified on-chain by source-available programs that anyone can read.

5 Ship real things

Devnet programs you can call today, an APK you can install today, SDKs you can npm install today. A manifesto without running code is a poem.

What Protocol 01 is

A privacy layer for Solana. Hold, send, and spend money without publishing your identity, your balance, or your transaction graph, while keeping the speed and cost of Solana underneath. One stack, several faces.

THE MOBILE APP

An Android wallet (Expo / React Native) with on-device proof generation, biometric unlock, shielding, private transfers, subscriptions, and unshielding. Privacy that fits in a pocket.

THE CHROME EXTENSION

The same shielded engine in a browser (Manifest V3), with license-key gated tiers and pairing with the phone by QR scan.

THE WEB APP

Next.js site at protocol-01.dev: docs, downloads, demos, weekly build-in-public videos.

THE SDKS

A family of TypeScript packages on npm so any developer can add private payments to their own product: zk-sdk (shielded pools), zkspl-sdk (confidential balances), specter-sdk (post-quantum stealth addresses), auth-sdk, merchant-sdk (subscriptions and license verification without PII), privacy-toolkit, p01-js, and more.

THE PROGRAMS

Around 14 Anchor programs on Solana devnet: the shielded pool, the zkSPL confidential-balance program, the specter stealth-address transport, subscriptions, streaming payments, the on-chain relay, the STARK verifier, the quantum vault, the service registry, the fee splitter, liquidity.

How it works: the journey of a note

The core object is a note. Everything else is proving things about notes without revealing them.

- SHIELD** **A note is born.** Your device creates a secret note and publishes only its commitment, a Poseidon hash of the note's secrets. The commitment goes into a Merkle tree on-chain. The chain knows a deposit happened, but not which future spend it belongs to.
- SPEND** **Three proofs in one.** Your device proves in zero knowledge that your note is in the tree (Merkle inclusion), that you know its secrets, and that you have never spent it before (a nullifier, a one-way tag the program records to prevent double spends). The verifier learns that some valid note was spent. It cannot tell which one. In a pool of fixed denominations, every note looks identical, so the anonymity set is the whole pool.
- TRANSFER** **Note to note.** Spending one note privately mints a commitment for the recipient. Value moves without any public link between sender and receiver. Subscriptions prove note ownership to a merchant program: the merchant sees a payment, never a wallet.
- UNSHIELD** **Back to public.** A note converts to public SOL at any address, with a relayer submitting the transaction so your public wallet never signs it.
- RECEIVE** **Stealth by default.** Specter stealth addresses combine ECDH with ML-KEM, a NIST post-quantum key encapsulation, and announce payments through chunked on-chain program accounts. A sender derives a one-time address only the recipient can detect and spend. A quantum adversary replaying the chain later still cannot link it.
- PROVE** **SNARKs to STARKs.** Proofs are moving from Groth16 (elliptic curves, trusted setup) to STARKs: a Winterfell prover over the Goldilocks field with Poseidon AIR, verified on-chain by our own multi-circuit FRI verifier. STARKs need no ceremony and rest on hash functions, the piece of cryptography quantum computers only dent instead of shatter. The prover compiles to WASM and runs on the phone.
- RELAY** **Honest necessities.** An on-chain relayer program with independent relay nodes so users do not doxx themselves paying fees, a refund keeper, event-scrubbed programs so even metadata leaks less, and a registry for services that accept shielded payments.

The circuits

Five production Circom circuits, each one a small mathematical contract. A proof convinces the on-chain verifier that the contract holds, without revealing the inputs.

CIRCUIT	MODEL	CONSTRAINTS	CAPACITY	STATUS
denominated_pool	Fixed-denomination shielded pool (Tornado Cash model). All notes in a pool share one value, so every withdrawal is indistinguishable.	~4,273	32K notes (depth 15)	ACTIVE
denominated_transfer	P2P note transfer inside a pool. Spends one note, mints a fresh commitment for the recipient. Maturity delay always enforced.	~4,400	32K notes	ACTIVE
confidential_balance	zkSPL account model. One circuit handles deposit, withdraw, send, and receive via selector flags, with a conservation law binding old and new balances.	1,382	account-based	ACTIVE
balance_proof	Range proof: committed balance meets a threshold. Lets you prove solvency or eligibility without showing the number.	644	account-based	ACTIVE
transfer	Zcash-style 2-in-2-out shielded transfer with arbitrary amounts and full Merkle inclusion.	~3.4M	1M notes (depth 20)	LEGACY

The two formulas that carry everything

```
commitment = Poseidon(nullifier_preimage, secret, deposit_epoch, token_mint)
nullifier   = Poseidon(nullifier_preimage, secret)
```

The commitment is what the chain stores: it hides the note behind a hash. The nullifier is what you reveal when spending: it is derived from the same secrets, but the chain cannot connect the two without inverting Poseidon. Publish the nullifier once and the note is dead forever, which is what makes double spending impossible.

zkSPL conservation law

```
old_balance + private_credit + public_credit == new_balance + private_debit +
public_debit
```

Every confidential balance update must satisfy this equation inside the proof. Money cannot be created or destroyed, only moved, and the amounts stay hidden behind Poseidon commitments salted per update.

Groth16 proving keys come from a Powers of Tau ceremony (pot20, 1.2 GB). This is exactly the trusted-setup dependency the STARK migration removes: the Winterfell prover needs no ceremony at all.

The proof pipeline

What actually happens between the tap and the confirmation. Seven steps, and the secrets never leave the device.

1 NOTE GENERATION – on device

The app draws fresh randomness for the note secrets (secret, nullifier preimage) and computes the Poseidon commitment locally with poseidon-lite. Nothing has touched the network yet.

2 MERKLE PATH – rebuilt from events

To spend, the circuit needs the note's path to the current Merkle root. The client rebuilds the tree from on-chain LeafInserted events, so it never has to ask a server "where is my note", which would leak exactly what privacy is supposed to hide.

3 WITNESS – private inputs assembled

Secrets, Merkle path, epochs, and amounts are packed into the circuit's witness. This is the sensitive object: it exists only in device memory, for seconds.

4 PROVE – WASM prover on the phone

The prover runs as WebAssembly inside the app (snarkjs for Groth16 circuits, Winterfell for STARK). Out comes a proof plus public inputs: root, nullifier, new commitment. The witness is discarded. The proof reveals nothing beyond "the contract holds".

5 RELAY – no signature from your wallet

The proof goes to an independent relay node (Railway, Fly.io), which submits it through the on-chain relayer program and pays the transaction fee. Your public wallet never signs, so the transaction cannot be attributed to you by its fee payer.

6 VERIFY – on-chain, trustless

The Solana program verifies the proof natively: Groth16 through alt_bn128 syscalls, STARK through the multi-circuit FRI verifier program. It checks the Merkle root against recent history, rejects any already-seen nullifier, then records the nullifier and inserts the new commitment.

7 SETTLE – the tree grows

The new commitment becomes a leaf, the root updates, and every client watching events can extend its local tree. Your note is now one anonymous leaf among thousands. Total time on a recent phone: a few seconds.

The architecture

One monorepo (pnpm + Turborepo, strict TypeScript), every layer open. This is the whole machine.

```
Protocol-01/
├─ apps/
│  ├─ mobile/      Android wallet – Expo / React Native, WASM prover in WebView
│  ├─ extension/   Chrome MV3 – shielded engine, license tiers, QR pairing
│  └─ web/         Next.js – protocol-01.dev, docs, downloads, demos
├─ packages/      13 TypeScript SDKs on npm
│  ├─ zk-sdk/      shielded pools: shield, transfer, unshield
│  ├─ zkspl-sdk/   confidential balances (zkSPL account model)
│  ├─ specter-sdk/ post-quantum stealth addresses (ECDH + ML-KEM)
│  ├─ merchant-sdk/subscriptions + license verification, zero PII
│  └─ ...          auth-sdk, p01-js, privacy-toolkit, ui, react-native-zk
├─ programs/      ~15 Anchor programs, Solana devnet
│  ├─ zk_shielded, p01_zkspl, specter, subscription, stream
│  ├─ p01_relayer, p01_stark_verifier, p01_quantum_vault
│  └─ p01_registry, p01-fee-splitter, p01_liquidity, whitelist
├─ circuits/      5 Circom circuits + Poseidon/Merkle templates, 73 tests
├─ stark/         Winterfell STARK prover – Poseidon AIR, Goldilocks, WASM
└─ services/      Rust prover service, relay nodes, refund keeper
```

How the layers talk

Apps call SDKs. SDKs build witnesses, run provers, and talk to programs. Programs verify proofs and hold the trees, nullifier sets, and subscriptions. Relay nodes sit between SDKs and programs so no user wallet ever signs a private transaction. The same SDKs the apps use are the ones published on npm: there is no privileged internal API, a third-party developer has exactly the power the official apps have.

Infrastructure honesty

Relay nodes run on Railway and Fly.io with self-healing RPC failover. They are a convenience, not a trust anchor: a relayer that turns malicious can censor a transaction but can never steal a note, forge a proof, or deanonymize a user. Anyone can run one.

Using the app

The whole point: all the cryptography above disappears behind six taps. End to end, three to five minutes on a recent phone.

1

Install

Download the release-signed Android APK from protocol-01.dev (or the GitHub releases page) and install it. No store account, no waitlist.

2

Create a wallet

Sign in with Google (embedded wallet via Privy) or create a classic local keypair with a seed phrase. Biometric unlock protects the app either way. Keys live on the device.

3

Fund it

Receive some SOL: devnet faucet today, any wallet transfer tomorrow. This balance is still public, like any normal Solana wallet.

4

Shield

Tap Shield and pick a denomination (0.1 SOL to start). The phone generates the ZK proof on-device and submits it through a relay. The note now lives in your private vault, indistinguishable from every other note in the pool.

5

Spend privately

Subscribe to a demo merchant (Netflix, Spotify, YouTube, Disney+ are attested on-chain): the merchant sees a valid payment, never your wallet. Or send the note peer to peer to another user's private address. Recurring billing pulls from shielded notes automatically.

6

Unshield when you want out

Convert any note back to public SOL at any address you choose, fresh or existing. The relayer submits it, so the receiving address has no on-chain link to the wallet that shielded.

Developers get the same flow in code: npm install the SDK, call shield / transfer / unshield / subscribe, done. The Chrome extension mirrors the app for desktop, and pairs with the phone by QR scan so both share one private vault.

What we refuse to do

No custody. We never hold user funds.

No PII. Not even for license keys, which are anonymous tokens verifiable without identity.

No immunity claims. The STARK migration is still under audit and we say so.

No crime pitch. Fixed denominations, maturity delays, and open verifiable code are chosen so the system defends ordinary people, not launderers.

No waiting for permission. We build what money should have had from the start.

Where it stands

Live on Solana devnet, pre-mainnet by design, third-party audit pending. Built solo since January 2026.

#2

DEV3PACK WORLDWIDE
SOLANA TRACK

~15

ANCHOR PROGRAMS
ON DEVNET

13

TYPESCRIPT SDKS
ON NPM

1

BUILDER
SINCE JAN 2026

Submitted to Colosseum Frontier 2026. Presented at The Bridge Demo Day. Selected for the Starknet Privacy Buildathon, Paris.

One person, one stack, one belief: **the ledger should prove your transaction is valid, and know nothing else.**

That is Protocol 01.